Which provider drew this emoji?

Separating drawing style from file-format shortcuts
in vendor classification of emoji images

Adam Zeraiki Lina Oumhand Charles Montluc Erdem Sanver

Team *The Winners* · BSc in Artificial Intelligence, Data Analytics and Management Sciences, CentraleSupélec and ESSEC Business School

Course teacher: Titouan Rigaud · 28 October 2026

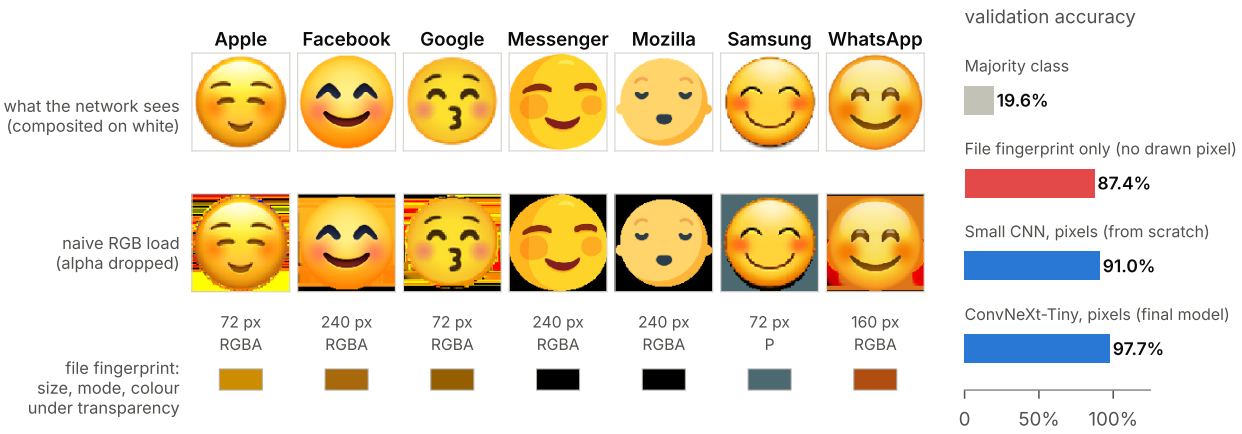


Figure 1. The same kind of emoji as drawn by the seven vendors of the competition. Top: the image every model in this paper receives (alpha-composited on white, upscaled here with nearest-neighbour so that the 72, 160 and 240 px native resolutions stay visible). Middle: the same files loaded with a naive RGB conversion, which drops the alpha channel and exposes the colour each encoder left under transparent pixels. Bottom: the file fingerprint (native size, PNG mode, mean hidden colour). Right: a random forest on the fingerprint alone, without reading a single drawn pixel, reaches 87.4% validation accuracy; our final pixel-only network reaches 97.7%.

Result at a glance

Final model: ConvNeXt-Tiny [1], ImageNet-1k weights, fine-tuned at 224 px, 5 folds, flip TTA	27.8 M parameters
Out-of-fold accuracy on all 9,879 training images (95% bootstrap CI)	**97.69%** [97.40, 97.98]
Kaggle accuracy of the final model, public / private leaderboard	0.98319 / **0.98218**
Team result: best selected entry, private leaderboard (rank among 18 teams)	0.98238 (5th)
Training cost of the final model on one RTX 4090	13 minutes, about $1

Abstract

We classify emoji images by the vendor that drew them (Apple, Facebook, Google, Messenger, Mozilla, Samsung or WhatsApp) on the course's Kaggle competition: 9,879 labelled training PNGs, 9,879 test PNGs, accuracy as the metric. Because every vendor draws the same Unicode concepts, this is a *style* recognition problem rather than an object recognition one. Before training any network we audited the files and found that the PNG container leaks the label: native size (72, 160 or 240 px), Samsung's palette mode and the colour stored under fully transparent pixels let a random forest reach 66.4% from size and mode alone and 87.4% with file statistics, without reading a drawn pixel. We removed this shortcut by design (alpha-compositing on white, one square canvas, exact-pixel augmentation) and measured on a fixed, duplicate-aware split how far the drawing alone can go. A ladder of models climbs from the team's September CNN (76.7%) to a four-stage BatchNorm CNN trained from scratch (91.0%) and to ImageNet-pretrained backbones fine-tuned end to end, where input resolution mattered more than architecture. Our final model, ConvNeXt-Tiny fine-tuned at 224 px in five folds with horizontal-flip test-time augmentation, reaches 97.69% out-of-fold accuracy on all 9,879 training images and 0.98218 on the private leaderboard. We also report a small but statistically clear finding: keeping the epoch with the lowest validation loss, the usual rule, costs 0.40 points (95% CI 0.22 to 0.59; exact McNemar $p = 2.4 \times 10^{-5}$) compared with keeping the last epoch of the OneCycle schedule, because validation loss and accuracy diverge once the network becomes over-confident; 96% of the final model's log loss comes from its 2.3% of errors. The remaining errors stay inside one export family and concentrate on documented look-alikes, such as WhatsApp's 2017 set, which Emojipedia describes as heavily inspired by Apple's.

Model chosen

Type: a convolutional neural network. The final model is **ConvNeXt-Tiny** [1] (27.8 M parameters), initialised from ImageNet-1k weights [2, 3] and fully fine-tuned on 224×224 inputs; five fold-models are averaged with horizontal-flip test-time augmentation. A 0.39 M-parameter CNN trained from scratch (EmojiCNNv2) serves as the reference for ablations and explainability.

Why not a vision-language model. All seven classes contain the same objects; the signal is how they are drawn (outlines, shading, anti-aliasing), which early convolutional filters capture [4] and which a model trained to describe image *content* is not built for. A CNN also trains in minutes, can be inspected (Grad-CAM, confusion structure) and reproduced exactly.

Why not hand-crafted features or frozen features. Colour histograms with HOG and an SVM reached 81.3%[R], below every CNN; ResNet18 used as a frozen feature extractor reached 77.5%[R] and 93.7%[R] once fine-tuned. Style lives below the object-level features that ImageNet training rewards; the backbone must be adapted.

Notation. Numbers marked [V] or untagged were produced by the submitted codebase and can be recomputed from its saved outputs; numbers marked [R] come from the team's 13 October experiment log or from Kaggle entries that the codebase does not reproduce.

Contents

1 Introduction 3

2 Background: the model families we used 4

2.1 Classical machine learning on hand-crafted features . . . 4
2.2 Convolutional neural networks . . . 4
2.3 Deeper CNNs: residual, efficient and modernised blocks . . . 5
2.4 Transformers and the Vision Transformer 5
2.5 Pretraining and transfer learning . . . 6
2.6 Combining models . . . 6

3 Data and preprocessing 7

3.1 The dataset . . . 7
3.2 What the files reveal before any pixel is read 7
3.3 Preprocessing recipe . . . 8
3.4 Data augmentation . . . 8
3.5 Duplicates and validation protocol . . . 9

4 Architecture 9

4.1 Reference model trained from scratch: EmojiCNNv2 . . . 9
4.2 Final model: ConvNeXt-Tiny, fine-tuned end to end . . . 10
4.3 Training recipe, ensemble and test-time augmentation . . . 11
4.4 A self-supervised transformer: DINOv2 . 11
4.5 Combining backbones: from voting to learned fusion . . . 12

5 Metrics chosen 12

6 Performance 13

6.1 How we got to the final model . . . 13
6.2 Experiment ladder . . . 13
6.3 Final model . . . 13
6.4 Which epoch to keep . . . 13
6.5 Is there a double descent? . . . 15
6.6 Did we overfit or underfit? . . . 17
6.7 Error analysis and explainability . . . 18
6.8 Kaggle leaderboard . . . 20
6.9 What did not work . . . 21

7 AI and code 22

7.1 Codebase and reproducibility . . . 22
7.2 Compute . . . 22
7.3 Use of AI tools . . . 22

8 Conclusion 22

A The team's 13 October experiment log 25

B Who tried what 26

C Per-vendor results of the CPU models 26

D Environment 26

1 Introduction

Every major platform ships its own drawing of the same Unicode emoji. A grinning face is a grinning face on every phone, yet an attentive reader can usually tell an Apple face from a Samsung one: the gradients, the outline weight, the shape of the eyes and the palette are a vendor's signature. The course challenge asks a model to do the same from a single PNG. This makes it a cousin of style recognition [5] and font identification [6], where the classes share content and differ in rendering, rather than of object recognition, where content is the class.

Such tasks are prone to shortcut learning [7]: a network rewarded only for accuracy will exploit any cue that correlates with the label, including cues that have nothing to do with the drawing. In this dataset the cue is the file itself. Each vendor exported its set through its own pipeline, and the pipeline left fingerprints in the container (Section 3.2). A related risk is leakage through duplicates [8, 9]: the competition data contain exact copies within the training set and across training and test. Both issues decide whether a validation number measures what we think it measures, so we treat them before modelling.

Contributions.

1. A file-level audit that quantifies the container shortcut: 87.4% accuracy without any drawn pixel, and a

preprocessing recipe that removes it while keeping the rendering intact (Section 3).

2. A controlled ladder of models on a fixed, duplicate-aware split, from the team's first CNN to a fine-tuned ConvNeXt-Tiny, with every verified number recomputable from saved probabilities (Section 6.2).
3. A final model at 97.69% out-of-fold accuracy and 0.98218 private leaderboard accuracy, trained in 13 minutes on one consumer GPU (Section 6.3).
4. Evidence that, at this accuracy, the rule used to pick a checkpoint matters: keeping the last epoch beats keeping the lowest-validation-loss epoch by 0.40 points, explained by over-confidence on a few hard images (Section 6.4).
5. A look at the non-monotonic validation curves through the lens of double descent, and a fitting diagnosis: the first CNN underfitted, the final model fits its training set but overfits only in its confidence, not its decisions (Sections 6.5 and 6.6).
6. An error analysis showing that the model separates export families from pixels alone and fails on documented look-alike designs (Section 6.7).

Outline. Section 2 introduces the families of models we used, from random forests to vision transformers. Section 3 describes the data and the preprocessing, Section 4 the architectures, Section 5 the metrics and Section 6 the results, starting with the path from our first model to the final one (Section 6.1). Section 7 covers the code and the use of AI, and Section 8 concludes.

2 Background: the model families we used

The project moved through most of the families of models used in computer vision today, from hand-crafted features to self-supervised transformers. This section introduces each one at the level needed to follow the rest of the paper; Table 1 summarises them and points to where each appears in our experiments.

2.1 Classical machine learning on hand-crafted features

Before deep learning, an image classifier had two separate parts: a human-designed feature extractor and a generic classifier. Histograms of oriented gradients (HOG) [10] describe local shape by the distribution of edge directions in small cells; colour histograms describe the palette. A support vector machine [11] then finds the maximum-margin boundary between classes, possibly in a higher-dimensional space reached through a kernel, and a random forest [12] averages many decision trees, each trained on a bootstrap sample with random feature subsets. Both are fast and interpretable, but they can only use the information the features expose. We used them twice: as a diagnostic of the file-format shortcut (a random forest on file metadata, Section 3.2) and as a hand-crafted baseline on pixels (HOG and colour histogram with an SVM, 81.28%).

2.2 Convolutional neural networks

A convolutional neural network (CNN) [13, 14] learns the features instead. A convolutional layer slides a small filter w of size $k \times k$ over the input and computes, at every position and for every output channel c',

$$y_{c'}(i,j) \;=\; b_{c'} + \sum_{c} \sum_{u,v=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} w_{c',c}(u,v)\, x_c(i+u,\, j+v), \tag{1}$$

so the same few weights are reused across the image (Fig. 2). Two assumptions are built in: what matters is local (locality), and a pattern means the same thing wherever it appears (translation equivariance). Pooling layers downsample the feature maps, so each later filter sees a larger part of the image (its receptive field), and stacking layers builds a hierarchy from edges to textures to parts. Batch normalisation [15] standardises activations within each mini-batch and makes deeper networks trainable at larger learning rates. These biases suit our task well: anti-aliasing, outline weight and shading are local, repeated patterns.

Table 1. Model families at a glance, with the best validation (or private leaderboard, †) accuracy each reached in this project. Parameter counts are for the seven-class versions, counted with `timm`.

Family	What it sees	Built-in assumption	Params	Best here
Random forest, SVM	hand-made features (file statistics, colour histogram, HOG)	the features are the right ones	n/a	87.37%
CNN from scratch	pixels, through small local filters	locality, translation equivariance	0.39 M	90.96%
Pretrained CNN (ResNet18, EfficientNet-B0, ConvNeXt-Tiny)	pixels, filters initialised on ImageNet	as above, plus ImageNet features transfer	4.0–27.8 M	97.69%
Vision transformer (DINOv2)	14×14 patches as tokens, global attention	almost none; learned from 142 M images	22.1 M (S/14)	0.95607†
Ensembles and fusion	the outputs of several models	errors of the members are not identical	sum	0.98238†

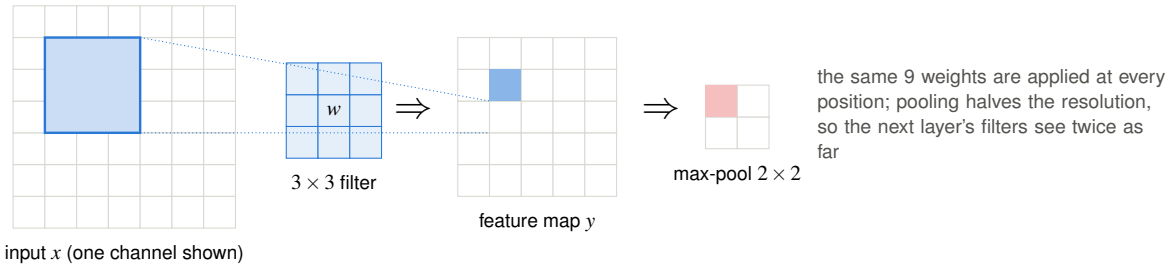


Figure 2. What a convolutional layer computes. A 3×3 filter (blue) slides over the input and produces one value of the feature map per position; max-pooling then keeps the strongest response in each 2×2 window.

2.3 Deeper CNNs: residual, efficient and modernised blocks

Simply stacking more layers eventually makes networks harder to optimise. ResNet [16] adds the input of each block to its output, $\boldsymbol{x} \leftarrow \boldsymbol{x} + F(\boldsymbol{x})$, so that a block only has to learn a correction and gradients flow through the identity path; ResNet18 stacks eight such blocks of two 3×3 convolutions. EfficientNet-B0 [17] is built from the inverted bottleneck (MBConv) of MobileNetV2 [18]: a 1×1 convolution expands the channels, a cheap depthwise 3×3 convolution filters each channel separately, a squeeze-and-excitation gate [19] reweights channels, and a 1×1 convolution projects back. ConvNeXt [1] revisits ResNet with design choices borrowed from transformers: a 7×7 depthwise convolution, LayerNorm instead of BatchNorm, a four-times expansion, GELU and fewer normalisation layers. Figure 3 compares the three blocks; ConvNeXt-Tiny became our final backbone.

2.4 Transformers and the Vision Transformer

The transformer [20] processes a sequence of tokens with self-attention: every token is projected to a query q, a key k and a value v, and each output is a weighted average of all values, with weights given by query-key similarity,

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V. \tag{2}$$

The Vision Transformer (ViT) [21] applies this to images by cutting them into fixed-size patches, embedding each patch as a token, adding a position embedding and a learnable class token, and passing the sequence through a stack of attention and MLP layers (Fig. 4). Unlike a CNN, a ViT assumes almost nothing about images: any patch can attend to any other from the first layer, and locality must be learned from data. This makes ViTs data-hungry, which is why they are almost always used after large-scale pretraining.

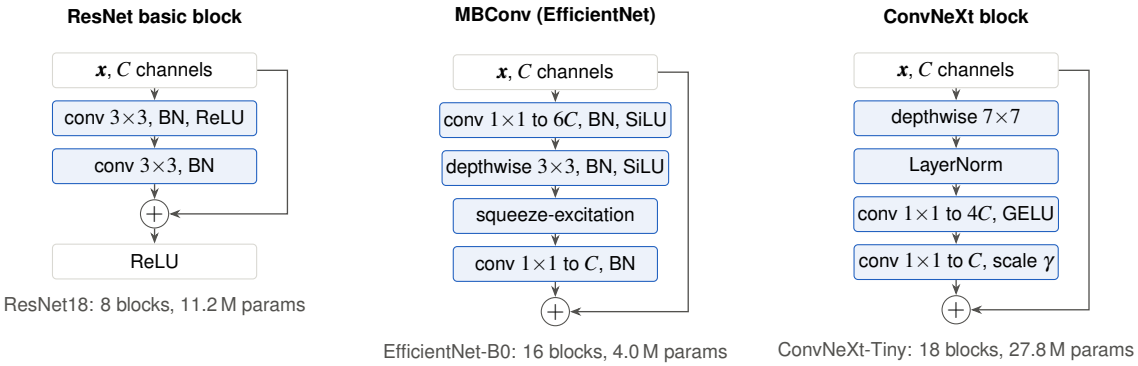


Figure 3. The three convolutional blocks we fine-tuned. All three add a learned correction to their input (the arrow on the right is the residual connection). They differ in how they filter: two dense 3×3 convolutions (ResNet), an expand, depthwise-filter and project sequence with channel gating (MBConv), or one large 7×7 depthwise filter followed by a transformer-style MLP (ConvNeXt).

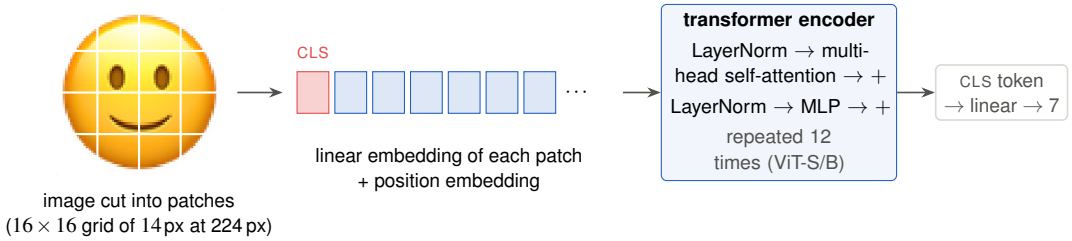


Figure 4. Vision Transformer. The image becomes a sequence of patch tokens; self-attention lets every patch interact with every other one in every layer, and the class token's final state is used for classification. DINOv2 models use 14-pixel patches.

2.5 Pretraining and transfer learning

Training a large network from scratch on 9,879 images would overfit. Transfer learning starts instead from weights learned on a much larger dataset and adapts them. With *supervised* pretraining, the network was trained to classify ImageNet's 1.28 million photographs into 1,000 classes [2]; all our pretrained CNNs come from this setting. With *self-supervised* pretraining, no labels are used at all: DINO [22] trains a student network to match the output of a teacher network, an exponential moving average of the student, on different augmented crops of the same image, and DINOv2 [23] scales this recipe to 142 million curated images. Either way, the pretrained network can be used *frozen*, as a fixed feature extractor with only a new linear head trained, or *fine-tuned*, with every weight updated at a small learning rate. Which of the two works depends on how close the pretraining task is to the new one; for vendor style, it turned out to be far (Section 6.1).

2.6 Combining models

An ensemble averages the predictions of several models; it helps when their errors differ [24, 25]. Soft voting averages probabilities with equal weights. Stacking [26] trains a second-level model on the first-level models' predictions, which must be made on data those models did not train on (out-of-fold predictions), otherwise the meta-model learns to trust overfitted members. A learned fusion goes one step further and combines the members' internal features rather than their final probabilities; we describe our version in Section 4.5.

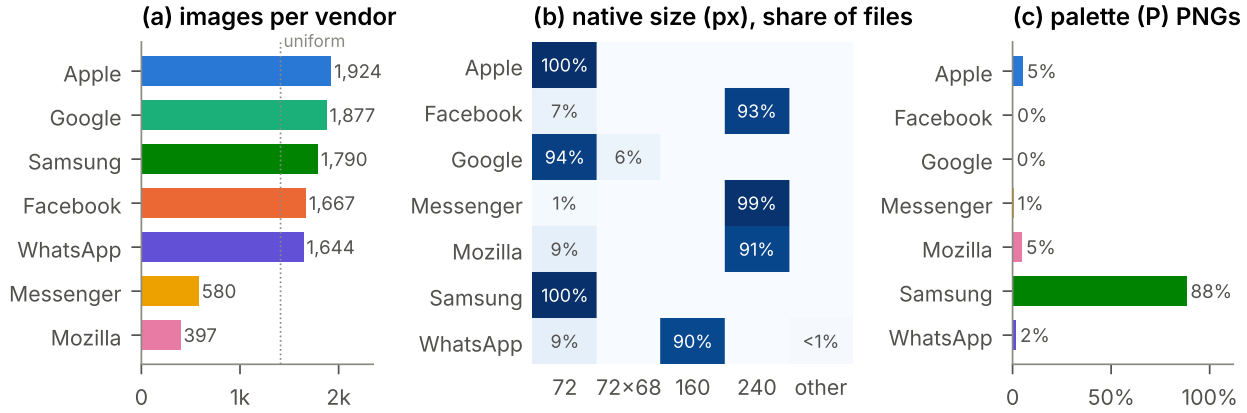


Figure 5. The training set and its file formats. (a) Images per vendor; the dotted line is a uniform split. (b) Native PNG size as a share of each vendor's files: three export families (72 px, 160 px, 240 px) almost coincide with groups of vendors. (c) Share of palette-mode PNGs: Samsung is the only vendor that saves most files as palette (P) images.

Table 2. File-level fingerprints per vendor, computed on the real training files[V]. The hidden colour is the mean RGB under fully transparent pixels, averaged over the vendor's files.

Vendor	Images	Modal size (share)	Modal mode (share)	Hidden colour (R, G, B)	Transparent
Apple	1,924	72 × 72 (99.9%)	RGBA (93.9%)	46.7, 39.1, 29.8	40%
Facebook	1,667	240 × 240 (93.0%)	RGBA (99.0%)	85.0, 68.1, 56.0	42%
Google	1,877	72 × 72 (93.6%)	RGBA (98.0%)	26.3, 21.9, 17.9	40%
Messenger	580	240 × 240 (98.6%)	RGBA (99.5%)	33.8, 29.4, 24.8	42%
Mozilla	397	240 × 240 (91.2%)	RGBA (95.2%)	5.5, 5.6, 5.5	37%
Samsung	1,790	72 × 72 (99.7%)	P (88.3%)	70.6, 96.0, 98.7	46%
WhatsApp	1,644	160 × 160 (90.3%)	RGBA (96.9%)	68.7, 54.7, 44.3	47%

3 Data and preprocessing

3.1 The dataset

The archive contains 9,879 training PNGs with labels and 9,879 unlabelled test PNGs. Classes are imbalanced (Fig. 5a): Apple has 1,924 images, Google 1,877, Samsung 1,790, Facebook 1,667 and WhatsApp 1,644, while Messenger has 580 (5.9%) and Mozilla 397 (4.0%). Predicting the majority class gives 19.5%; chance is 14.3%.

3.2 What the files reveal before any pixel is read

Table 2 and Figs. 5 and 6 summarise what the containers say. Apple, Google and Samsung files are 72 px wide, WhatsApp files 160 px, and Facebook, Messenger and Mozilla files 240 px. Samsung saved 88% of its files as palette PNGs, where every other vendor used RGBA. The RGB values stored under fully transparent pixels, invisible on any screen, differ by vendor too, because an encoder keeps whatever was in its buffer (Fig. 6a): near-black for Mozilla, one near-constant blue-grey for Samsung, a brownish mixture elsewhere. Messenger and Facebook share the 240 px export because Facebook replaced the Messenger-specific set with its platform-wide set in October 2017 [27]. The test set has the same mix: the total variation distance between the train and test distributions of size and mode is 0.5%.

We then measured how much these fingerprints are worth (Fig. 6b). Classifiers that never see a drawn pixel reach 64.6% (logistic regression) and 66.4% (random forest) from width, height, PNG mode and the alpha flag, 84.4% once file size and transparent fraction are added, and 87.4% with the hidden colour[V]. A network given the raw file would be free to learn this instead of the drawing, and its validation score would look excellent for the wrong reason. We therefore removed the shortcut (Section 3.3) and kept the metadata out of

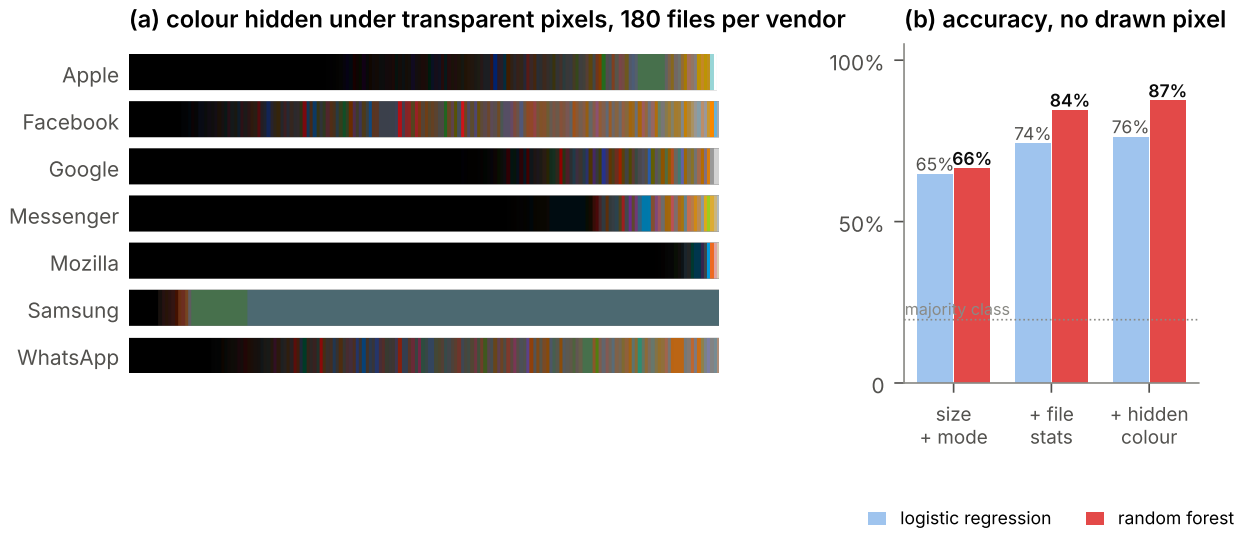


Figure 6. The container leaks the label. (a) Each strip shows the mean colour stored under fully transparent pixels for 180 random files of a vendor, sorted by luminance; these pixels are invisible when the image is displayed. Samsung's palette files share one blue-grey, Mozilla's are black. (b) Validation accuracy of models that see only file metadata (stored split, 1,979 images)[V]: up to 87.4% without a single drawn pixel.

the final model; Section 6.2 measures, as an ablation, what it would have added on top of pixels.

3.3 Preprocessing recipe

One function feeds every model in this paper. Let $F \in [0,255]^{H\times W\times 3}$ be the stored colour and $\alpha \in [0,1]^{H\times W}$ the alpha channel of a file. We compute

$$I \;=\; \alpha \odot F + (1-\alpha)\odot 255, \tag{3}$$

which sets every transparent pixel to white and therefore erases the hidden colour (Fig. 1, Fig. 7). The image is then fitted into a square canvas keeping its aspect ratio (Lanczos filter) and padded with white, because a few Google files are 72×68; resized to the model's input (96 px for networks trained from scratch, 224 px for ConvNeXt-Tiny); stored as `uint8` and cached; and normalised to $[-1,1]$ for networks trained from scratch or with the ImageNet mean and standard deviation for the pretrained backbone. We do not crop, do not convert to greyscale (palettes are part of a vendor's style) and do not equalise histograms. Resizing to a common canvas also removes the native size as a direct cue; whatever the network learns about it must come from the rendering.

The team measured the effect of compositing directly: the same three-block CNN scored 92.11% with a default RGB load and 93.62% with alpha compositing on white, everything else equal (E2 and E3[R]). The naive load also exposes the hidden colour, so the composited input wins even though it gives the network less to cheat with. The resampling filter matters for the same reason: E15, the team's best Kaggle entry, adds models trained at 144 and 216 px with nearest-neighbour resizing, which are exact $2\times$ and $3\times$ copies of a 72 px file with no interpolated pixel (Section 6.8).

3.4 Data augmentation

Training augmentation is limited to a horizontal flip and an integer translation of up to 4 px (white padding, random crop). Both are exact pixel operations: no interpolation touches the image, so the anti-aliased outlines and gradients that distinguish vendors stay exactly as exported. Rotation and rescaling resample the image and blur precisely that signal. The team tested it: on ConvNeXt-Tiny at 128 px, rotation and scale augmentation with label smoothing [28] scored 94.38% against 95.90% without[R], a loss of 1.5 points. At test time every model sees the image and its mirror and the two softmax outputs are averaged.

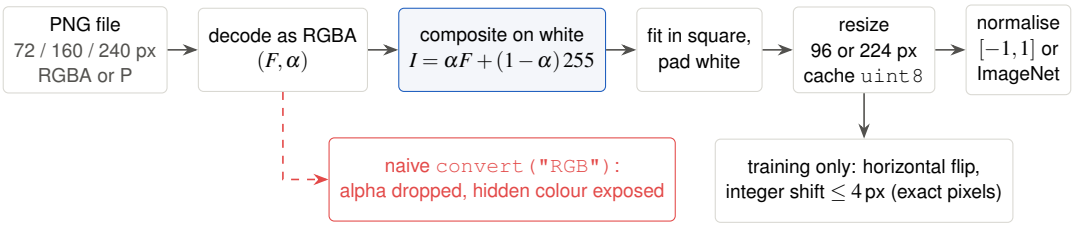


Figure 7. Preprocessing pipeline shared by every model. Compositing on white is the step that removes the hidden-colour shortcut of Fig. 6; the naive branch (red) is what a default RGB load does.

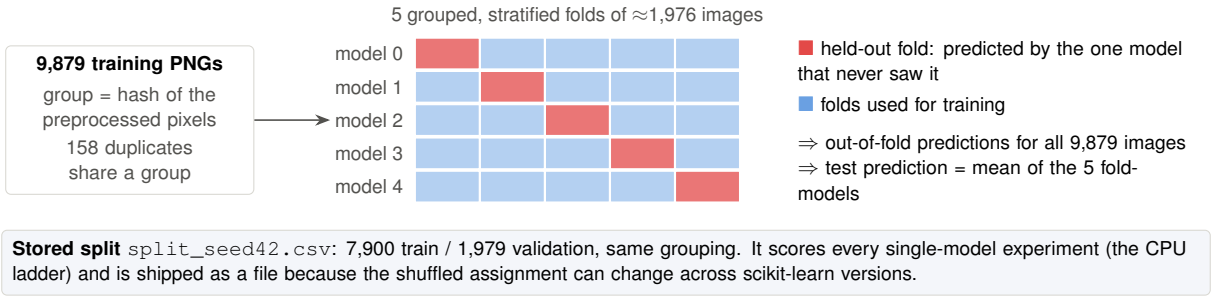


Figure 8. Validation protocol. Duplicates are grouped by pixel hash so that no image has a copy on the other side of a split. The final model is evaluated out-of-fold on all training images; its restriction to the stored validation split makes it directly comparable with the CPU models.

3.5 Duplicates and validation protocol

Hashing the preprocessed pixels shows that 158 training images are exact copies of another training image (149 before preprocessing), none with a conflicting label. The test set is not independent either: 333 test files are byte-identical to a training file and 163 test images duplicate another test image. A random split would place copies on both sides and inflate validation accuracy [9], so every split in this project is a `StratifiedGroupKFold` [29] with the pixel hash as group: identical images always land on the same side, and each fold keeps the class proportions (Fig. 8).

Two protocols are used. A stored split of 7,900 training and 1,979 validation images scores every single-model experiment. The final model is trained in five folds and reported out-of-fold on all 9,879 images, and also on the subset that belongs to the stored validation set. Nothing was ever selected by looking at the test set. Two optional post-processing rules exist in the codebase (give a test file that is byte-identical to a training file that file's label; make identical test images share one prediction); both are off, and on the final predictions they would change 0 of 9,879 labels, because the model already gets those images right.

4 Architecture

4.1 Reference model trained from scratch: EmojiCNNv2

The team's first model (30 September) had three Conv–ReLU–MaxPool blocks without normalisation and was trained with Adam at a constant learning rate of 10^{-3} for 20 epochs. Re-scored with its saved weights on the stored split it reaches 76.71% (1,518 of 1,979)[V], with a training accuracy of only 75%: it was underfitting. EmojiCNNv2 keeps the philosophy (small, from scratch, fully explainable) and fixes what held it back: BatchNorm after every convolution, a fourth stage so that the receptive field covers the 96 px canvas, AdamW [30] with a OneCycle schedule [31], and the augmentation above (Table 3 and Fig. 9). It has 390,695 parameters, costs about 0.14 GFLOPs per image and trains at 143 s per epoch on two CPU cores.

Table 3. EmojiCNNv2, the reference CNN trained from scratch on 96×96 inputs.

Stage	Layers	Output ($C \times H \times W$)	Parameters
input	RGB, normalised to $[-1, 1]$	$3 \times 96 \times 96$	0
1	Conv 3×3 (3 → 32) + BN + ReLU, MaxPool 2	$32 \times 48 \times 48$	928
2	Conv 3×3 (32 → 64) + BN + ReLU, MaxPool 2	$64 \times 24 \times 24$	18,560
3	Conv 3×3 (64 → 128) + BN + ReLU, MaxPool 2	$128 \times 12 \times 12$	73,984
4	Conv 3×3 (128 → 256) + BN + ReLU	$256 \times 12 \times 12$	295,424
head	global average pooling, dropout 0.3, linear 256 → 7	7	1,799
total			390,695

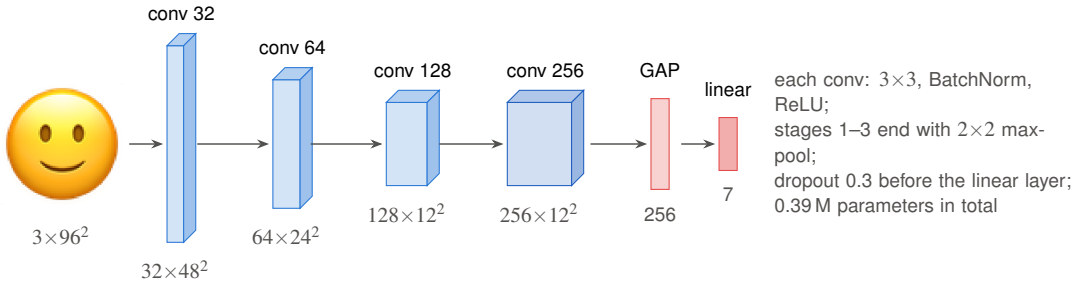


Figure 9. EmojiCNNv2. Spatial size halves and channel count doubles at each stage; global average pooling (GAP) turns the last $256 \times 12 \times 12$ feature map into one vector, and a linear layer gives the seven logits.

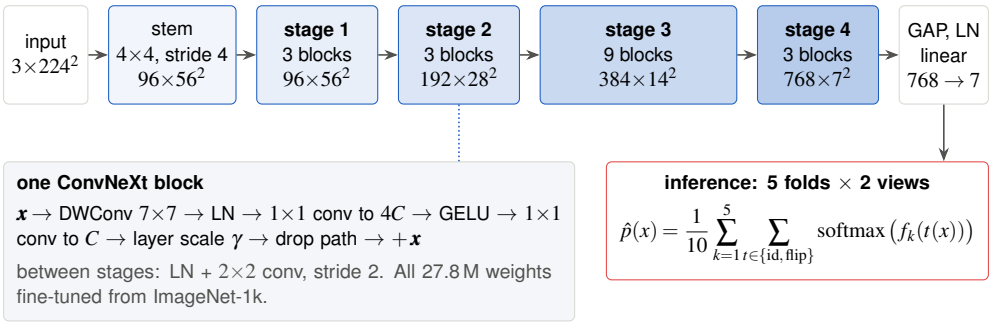


Figure 10. The final network. ConvNeXt-Tiny with a seven-way head. One copy is trained per fold; at test time the five fold-models each see the image and its horizontal mirror and the ten softmax vectors are averaged.

4.2 Final model: ConvNeXt-Tiny, fine-tuned end to end

ConvNeXt [1] is a purely convolutional network built with the macro-design of vision transformers (Fig. 10). A 4×4 convolution with stride 4 turns the 224×224 input into a 56×56 grid of 96-channel patches; four stages with 3, 3, 9 and 3 blocks and widths 96, 192, 384 and 768 follow, separated by downsampling layers (LayerNorm and a 2×2 convolution with stride 2). A block computes

$$\boldsymbol{x} \leftarrow \boldsymbol{x} + \mathrm{DropPath}\Big(\gamma \odot W_2\, \mathrm{GELU}\left(W_1\, \mathrm{LN}(\mathrm{DWConv}_{7\times7}(\boldsymbol{x}))\right)\Big), \tag{4}$$

with W_1 expanding to four times the width, W_2 projecting back, a learned layer scale γ and stochastic depth [32]. Global average pooling, a LayerNorm and a linear layer produce the seven logits; the 1,000-way ImageNet head is discarded. With our head the network has 27,825,511 parameters, all updated during fine-tuning from the ImageNet-1k weights distributed by `timm` [3].

The team's experiment series explains the choice of this architecture at this resolution. Generic ImageNet features are not enough when frozen (ResNet18 [16] as a fixed extractor: 77.48%^{R}). Fine-tuning the whole backbone adapts the filters to rendering style (the same ResNet18 fine-tuned: 93.72%^{R}). ConvNeXt-Tiny

Table 4. Training recipes of the reference CNN and of the final model.

	EmojiCNNv2 (reference)	ConvNeXt-Tiny (final)
Input	96×96, $[-1, 1]$	224×224, ImageNet mean and std
Initialisation	random	ImageNet-1k (`convnext_tiny.fb_in1k`)
Optimiser	AdamW, lr 2×10^{-3}, weight decay 10^{-4}	AdamW, lr 2×10^{-4}, weight decay 0.05
Schedule	OneCycle, 25 epochs, batch 64	OneCycle, 10 epochs per fold, batch 32
Regularisation	BatchNorm, dropout 0.3, flip + shift	stochastic depth 0.1, flip + shift
Loss	cross-entropy	cross-entropy, no label smoothing
Checkpoint kept	lowest validation loss	last epoch (run B, final); lowest validation loss (run A, ablation)
Validation	stored split (7,900 / 1,979)	5-fold grouped CV, out-of-fold on 9,879
Inference	single pass	mean of 5 models $\times$ {image, mirror}
Hardware	2 CPU cores, fp32	NVIDIA RTX 4090 (RunPod), mixed precision

beat ResNet18 and EfficientNet-B0 [17] at the same 128 px input (95.90% against 93.72% and 93.12%^{R}), which we attribute to its large depthwise kernels and LayerNorm capturing stroke and shading over a wider context. Input resolution then gave the largest single gain (97.57% at 224 px^{R}): 224 px is the pre-training resolution and is close to the 240 px native size of the Facebook, Messenger and Mozilla files, which are therefore barely resampled.

4.3 Training recipe, ensemble and test-time augmentation

Table 4 gives both recipes. The final model is trained with AdamW (learning rate 2×10^{-4}, weight decay 0.05) under a OneCycle schedule over 10 epochs with batches of 32, stochastic depth 0.1, plain cross-entropy and mixed precision [33]. Seeds are 42 plus the fold index with deterministic cuDNN kernels. One model is trained per fold and validated on the fold it never saw; the weights kept are those of the last epoch (Section 6.4 shows why). Predictions are averaged in probability space (soft voting) [25], so that one confident fold cannot be outvoted by three unsure ones.

4.4 A self-supervised transformer: DINOv2

Besides convolutional networks, the team tried a vision transformer [21]. DINOv2 [23] is a ViT pretrained without labels on LVD-142M, a curated set of about 142 million images, by self-distillation: a student network learns to reproduce a teacher's output on different crops of the same image. With a patch size of 14, a 224×224 input becomes a 16×16 grid of tokens processed by global self-attention. Its features are among the strongest general-purpose ones available, which made it the natural candidate for a model that needs no task-specific design. Two entries used it (Table 10): Lina fine-tuned DINOv2 on the training set (private score 0.95607), and an ensemble of a DINOv2 model with a ConvNeXt model scored 0.96477. Both trail ConvNeXt-Tiny alone (0.98218), and the ensemble shows that adding the transformer to a convolutional network pulled it down. These runs were made outside the submitted codebase and their configuration (model size, input resolution, epochs, which layers were trained) was not recorded with them, so we report their scores without a validation estimate and cannot say how much tuning would have closed the gap.

We see two plausible reasons for the gap, neither of which we tested. First, the DINO objective that DINOv2 builds on trains features to be invariant to colour jittering, Gaussian blur and solarization [22]: precisely the palette, gradient and anti-aliasing cues that separate vendors, so fine-tuning must first undo an invariance that pretraining worked to build. Second, geometry: a 72 px emoji upscaled to 224 px puts about 4.5 native pixels in each 14 px patch, and the first layer of a ViT mixes outline, fill and edge pixels into one token, whereas ConvNeXt's 4×4 stem covers about 1.3 native pixels and its 7×7 depthwise kernels keep strokes separate in the early layers. For style recognition at this scale, the convolutional inductive bias appears to be an advantage rather than a limitation.

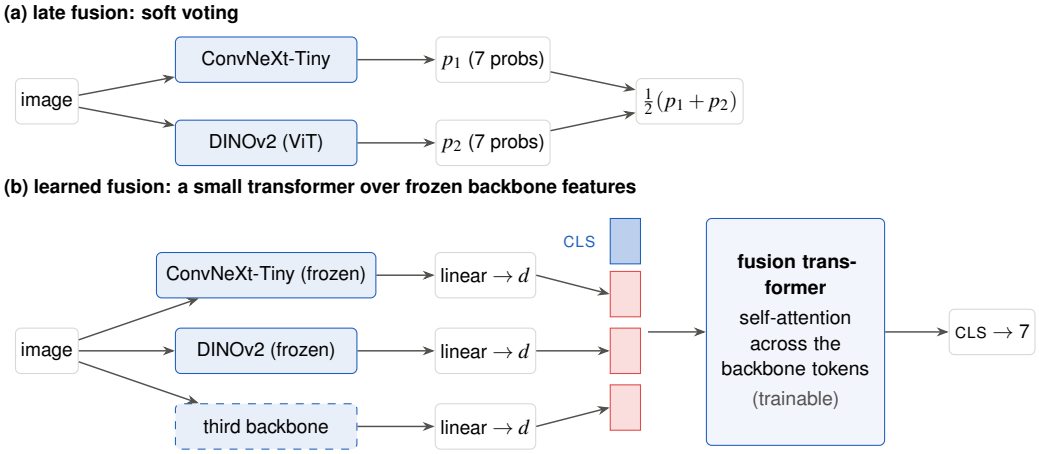


Figure 11. Two ways to combine different backbones. (a) Averaging the class probabilities, which is what our final ConvNeXt ensemble does across folds. (b) Learned fusion: each backbone's feature vector becomes one token, and a small trainable transformer learns how to weigh and combine them. The dashed box is the third member of the team's experiment (see the note below).

4.5 Combining backbones: from voting to learned fusion

Our final model combines members of the same architecture by soft voting (Fig. 10). The team also explored combining *different* backbones, on the logic of Section 2: an ensemble helps only if its members make different mistakes, and a self-supervised transformer and a supervised CNN see an image very differently. Figure 11 shows the two ways to combine them. In late fusion (soft voting) each backbone produces its own class probabilities and these are averaged. In learned fusion, each frozen backbone produces a feature vector, a linear layer projects it to a common width so that it becomes one token, and a small trainable transformer attends across the tokens, in effect translating each backbone's representation into a shared space before a class token makes the decision. Only the projection layers and the fusion transformer are trained, which keeps the number of trainable parameters small, and as with stacking the fusion model must be trained on data the backbones were not fine-tuned on.

The only recorded result of this line of work is the DINOv2 + ConvNeXt entry in Table 10: 0.96477 on the private leaderboard, against 0.98218 for ConvNeXt-Tiny alone. Whatever the combiner, adding a member that is clearly weaker on this task pulled the stronger model down, which is the outcome the ensemble literature predicts when one member's errors dominate.

To complete before submission (Adam)

The configuration of the three-backbone fusion experiment was run outside the submitted codebase and is not recorded in our files: which third backbone was used, whether the transformer fused features or probabilities, its size and training data, and whether the 0.96477 entry came from the fusion model or from a vote. Fill these in from the Codex session (see `members/adam-zeraiki/README.md` in the repository), or delete this box and the dashed member of Fig. 11(b).

5 Metrics chosen

Accuracy is the competition metric and our headline number, but alone it hides two things we care about. We therefore report:

- **Accuracy with its uncertainty.** With n images the binomial standard error is $\sqrt{a(1-a)/n}$: about 0.65 points at 91% and 0.4 points at 97% on the 1,979-image split, 0.15 points out-of-fold on 9,879 images. We add 95% bootstrap intervals (2,000 resamples) for the final model.
- **Macro F1 and balanced accuracy** (mean per-class recall). Messenger and Mozilla together are 9.9% of the data; these metrics make a drop on a small class visible.

- **Per-class recall and the confusion matrix**, because the errors have structure (export families, look-alike vendors).
- **Log loss** $-\frac{1}{n}\sum_i \log \hat{p}_{i,y_i}$ and the **expected calibration error** $\mathrm{ECE} = \sum_b \frac{|B_b|}{n} |\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|$ over 15 confidence bins [34], to see whether the probabilities are trustworthy.
- **Paired tests.** When two models are scored on the same images we report the counts of images only one of them gets right and an exact McNemar test [24, 35], because two accuracies alone do not say whether the models fail on the same images.

6 Performance

6.1 How we got to the final model

The final model was not designed in one step. Table 5 tells the project in the order it happened: each row is something we tried, what it gave, and what that taught us, which is what decided the next row. The complete list of runs with their settings is in Appendices A and B.

Three turns shaped the project. The first was finding the file-format shortcut (step 3): it changed the question from "how high can we score" to "how high can we score from the drawing", and every later comparison was made under that rule. The second was the failure of frozen features (step 5), which showed that vendor style is not what ImageNet pretraining encodes, and that the backbone has to be adapted. The third was the gap between validation loss and accuracy at the end (step 11), which changed how we choose a checkpoint. Several ideas did not work, and they are as informative as the ones that did: resampling augmentation, retraining on all data, and adding a weaker transformer to a strong CNN all lost or failed to gain accuracy (Section 6.9).

6.2 Experiment ladder

Table 6 and Fig. 12 list every model of the project; each rung changes one thing. The file metadata alone set a high bar: 87.4% without seeing the drawing, which any pixel model should clear before it can claim to have learned style. The September CNN does not (76.71%). EmojiCNNv2 reaches 90.96% with BatchNorm, a fourth stage, OneCycle and exact-pixel augmentation and nothing else; on the same 1,979 images it is right on 346 images the baseline gets wrong and wrong on only 64 that the baseline gets right (exact McNemar $p < 10^{-40}$). Adding the eight file-format features by late fusion gives 93.38% (79 images right only with fusion, 31 only without; $p = 5.3 \times 10^{-6}$). That 2.4-point gain is the price of keeping the final model pixel-only; we consider it well spent, since it is the part of the score that comes from the encoder rather than the artist. The pretrained backbones then take over, with resolution as the decisive variable.

6.3 Final model

The final model reaches an out-of-fold accuracy of **97.69%** (9,651 of 9,879 images; 95% bootstrap CI [97.40, 97.98]), balanced accuracy 0.9762, macro F1 0.9773 and log loss 0.1105[V]. Restricted to the stored validation split it scores 97.07% (1,921 of 1,979), the number to compare with the CPU rows of Table 6. The five folds range from 97.47% to 97.87% (Table 7), the spread expected from folds of about 1,976 images. Recall is above 96% for every vendor (Table 8); the smallest class, Messenger, is among the best recognised (98.8%).

6.4 Which epoch to keep

Our first five-fold run (run A) followed the usual rule and kept, for each fold, the epoch with the lowest validation loss. It chose epochs 7, 4, 6, 5 and 8, while validation accuracy was still rising at epoch 10 (Fig. 13). Run B kept the last epoch of the OneCycle schedule, where the learning rate has been annealed to zero; this rule involves no selection on the validation set at all. Run B re-trained the five folds from the same seeds on the same GPU, and the two runs reproduced each other to the last decimal at every epoch, so the difference between them is entirely the epoch that was kept. Run B is better by 0.40 points out-of-fold

Table 5. From first try to final model. Accuracies are validation accuracies unless marked † (private leaderboard). [V] verified by the codebase, [R] from the team's log or Kaggle.

	What we tried	Result	What we learned, and what we changed
1	First CNN: 3 conv blocks, no normalisation, Adam at a constant 10^{-3}, 20 epochs (30 Sept.)	76.71%[V]	Training accuracy only 75%: the model underfits. **Next:** BatchNorm, a fourth stage and a OneCycle schedule.
2	EmojiCNNv2 (0.39 M parameters), same data	90.96%[V]	+14 points from optimisation and normalisation alone, no extra data. A good CNN from scratch is a strong baseline.
3	File metadata only (size, PNG mode, hidden colour), random forest	87.37%[V]	The file format leaks the label. **Next:** composite on white, square canvas, exact-pixel augmentation; keep the final model pixel-only. Same CNN with compositing: 92.11% → 93.62%[R].
4	Hand-crafted features: colour histogram + HOG, SVM	81.28%[R]	Below every trained CNN: the cues are too fine for generic descriptors. **Next:** learn the features.
5	Pretrained ResNet18 as a frozen feature extractor, 128 px	77.48%[R]	ImageNet features describe objects, not drawing style. **Next:** fine-tune the whole network.
6	Full fine-tuning at 128 px: ResNet18, EfficientNet-B0, ConvNeXt-Tiny; ResNet18 at 160 px (Charles)	93.72%, 93.12%, 95.90%[R]; 0.96396†	Fine-tuning recovers 16 points; ConvNeXt is best. **Next:** build on ConvNeXt-Tiny.
7	Stronger augmentation: rotation, scaling, label smoothing	94.38%[R]	−1.5 points: resampling blurs the outlines that identify a vendor. **Next:** flips and integer shifts only.
8	Input resolution 128 → 224 px	97.57%[R]	+1.7 points, the largest single gain: resolution matters more than architecture.
9	Ensembles (128 + 224 px), retraining on all data, nearest-neighbour upscaling (E15, Erdem)	97.93%[R]; 0.98238†	Retraining on all data changed the private score by +0.14 and −0.08 points: no reliable gain. Exact integer upscaling gave the team's best entry.
10	Self-supervised ViT: DINOv2 fine-tuned (Lina); DINOv2 + ConvNeXt ensemble	0.95607†, 0.96477†	The transformer is weaker here, and adding it pulls ConvNeXt down. **Next:** combine only comparably strong members.
11	ConvNeXt-Tiny, 5 grouped folds, flip TTA, lowest-validation-loss epoch (run A)	97.29%[V]	Validation loss and accuracy disagree. **Next:** keep the last epoch of the schedule.
12	Same, last epoch (run B): **final model**	97.69%[V]; 0.98218†	+0.40 points over run A ($p = 2.4 \times 10^{-5}$), 13 minutes of training.

(97.69% against 97.29%; 95% paired bootstrap CI [0.22, 0.59]); on the same 9,879 images 64 are right only under run B and 24 only under run A (exact McNemar $p = 2.4 \times 10^{-5}$).

The reason is over-confidence [34]. Once almost every image is right, the cross-entropy is dominated by a handful of confident mistakes: in the final model 96% of the total log loss comes from the 2.3% of images it gets wrong. Later epochs push every probability towards one; this fixes borderline images (accuracy rises) and makes the remaining mistakes more confident (loss rises). Figure 14 shows both effects. Run B's mean confidence on its errors is 0.86 against 0.77 for run A, and 71 of its errors are made with more than 99% confidence against 40; its calibration error is slightly higher (ECE 1.61% against 1.24%), yet it is more accurate at every coverage level. For a metric that only counts correct labels, the last epoch is the right choice. We also averaged both snapshots of every fold, ten checkpoints in all, in the spirit of weight or snapshot averaging [36]: 97.70% out-of-fold, a tie with run B, and slightly lower on both leaderboards

Table 6. Validation results of every model. Verified rows (V) were produced by the submitted codebase on the stored split or out-of-fold (OOF); reported rows (R) come from the team's log on an 80/20 split of about 1,976 images and are only roughly comparable. *Macro precision, the only macro metric in that log.

Model	Input	Accuracy	Macro F1	
Majority class (always Apple)	none	19.61%		V
File format only: size, mode, alpha flag (random forest)	no pixels	66.40%		V
File metadata incl. hidden colour (random forest)	no drawing	87.37%	0.867	V
Colour histogram + HOG, RBF SVM (E1b)	64 px	81.28%	0.848*	R
Baseline CNN, 30 September (3 blocks, no BN, Adam)	96 px	76.71%	0.738	V
Simple CNN, white compositing (E3)	96 px	93.62%	0.933*	R
EmojiCNNv2, pixels only	96 px	90.96%	0.907	V
EmojiCNNv2 + 8 file-format features (late fusion, ablation)	96 px	93.38%	0.926	V
ResNet18, frozen features (E4)	128 px	77.48%		R
ResNet18, fine-tuned (E5)	128 px	93.72%		R
EfficientNet-B0, fine-tuned (E6)	128 px	93.12%		R
ConvNeXt-Tiny, fine-tuned (E6b)	128 px	95.90%		R
ConvNeXt-Tiny + rotation/scale aug. + label smoothing (E7)	128 px	94.38%		R
ConvNeXt-Tiny, fine-tuned (E8)	224 px	97.57%		R
Ensemble ConvNeXt-Tiny 128 + 224 px (E9)	both	97.93%		R
ConvNeXt-Tiny, 5 folds, TTA, lowest-val-loss epoch (run A, ablation)	224 px	97.29% OOF	0.972	V
ConvNeXt-Tiny, 5 folds, TTA, last epoch (run B): final model	224 px	**97.69%** OOF	**0.977**	V
same model, restricted to the stored 1,979-image split		97.07%	0.969	V
Both checkpoints of every fold averaged (10 checkpoints, ablation)	224 px	97.70% OOF	0.977	V

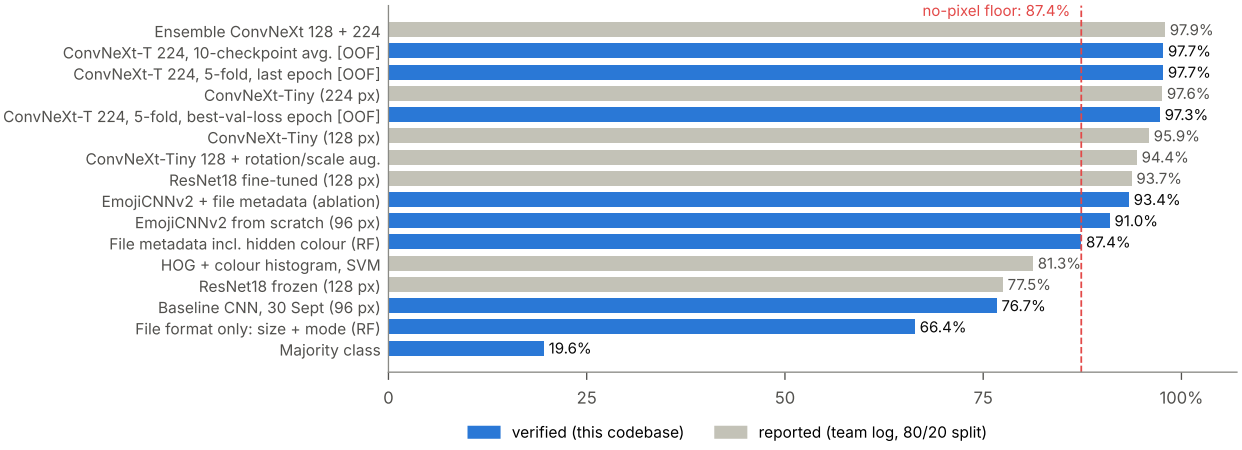


Figure 12. The experiment ladder. Validation accuracy of every model in the project, sorted. Blue bars were produced by the submitted codebase (stored split or out-of-fold), grey bars come from the team's log on a different 80/20 split. The dashed line is the accuracy a random forest reaches from file metadata alone: the September CNN, the HOG + colour SVM and frozen ResNet18 features sit below it; every BatchNorm or fine-tuned CNN is above.

(Section 6.8). Giving half the vote to less-converged weights does not help, so the final model is the five last-epoch fold-models: the simpler rule and the better number.

6.5 Is there a double descent?

Classical learning theory predicts a U-shaped test error: it falls while a model fits the signal and rises once it starts to fit noise. Belkin et al. showed that beyond the interpolation threshold, the capacity at which a model fits its training set exactly, test error can fall a second time, a shape they called double descent [37].

Table 7. Per-fold results (flip TTA). Both runs share every training batch; only the epoch kept differs.

	Run A (lowest val. loss)		Run B (last epoch)	
Fold	Accuracy	Epoch	Accuracy	Min.
0	97.67%	7	97.87%	2.9
1	96.76%	4	97.47%	2.4
2	97.42%	6	97.67%	2.7
3	96.81%	5	97.62%	2.6
4	97.77%	8	97.82%	2.4
OOF	97.29%		**97.69%**	13.0

Table 8. Per-vendor metrics of the final model, out-of-fold.

Vendor	Prec.	Recall	F1
Apple	0.952	0.967	0.960
Facebook	0.991	0.977	0.984
Google	0.968	0.987	0.977
Messenger	0.979	0.988	0.984
Mozilla	0.982	0.962	0.972
Samsung	0.984	0.990	0.987
WhatsApp	0.992	0.964	0.978

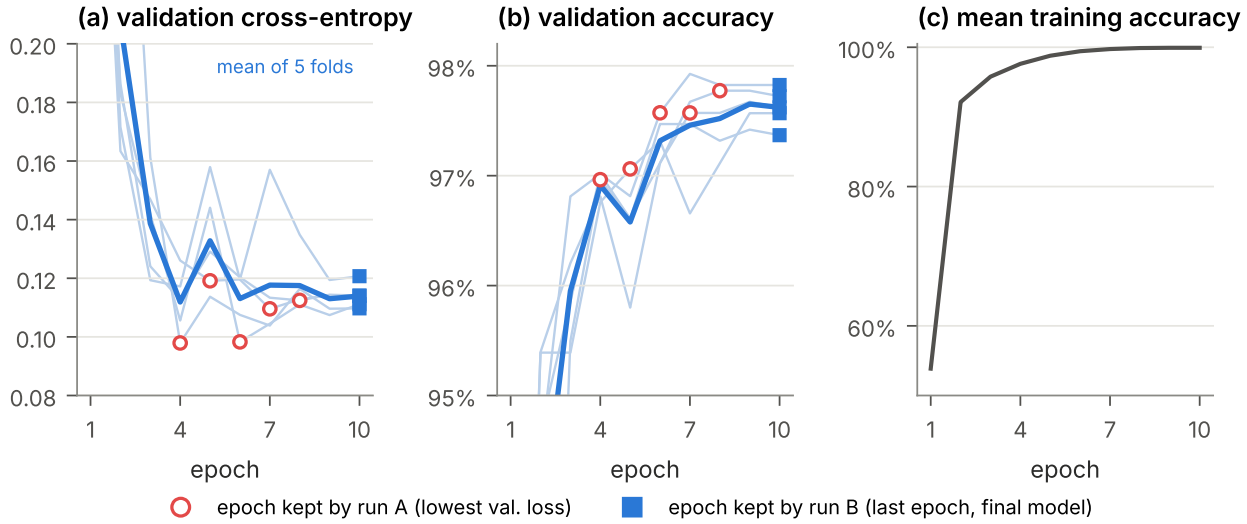


Figure 13. Validation loss and accuracy disagree on when to stop. Thin lines are the five folds, the thick line their mean. (a) Validation cross-entropy is lowest around epochs 4 to 8, where run A stopped (circles). (b) Validation accuracy keeps improving until the end of the schedule, where run B stopped (squares). (c) Training accuracy saturates at 99.9% by epoch 8: the network is still sharpening its decisions, not learning new ones.

Nakkiran et al. found it in deep networks both as a function of model size and of training time: in *epoch-wise* double descent, test error peaks when a model has been trained just long enough to reach near-zero training error and falls again if training continues, most strongly when training labels are noisy [38].

Our curves contain two non-monotonic shapes (Fig. 15). The clearest is in EmojiCNNv2 (panels a and b). Validation loss falls for three epochs, climbs from 1.16 to 2.15 by epoch 7 while validation accuracy drops from 55% at epoch 2 to 43%, then descends a second time to 0.31; training loss decreases at every epoch. It looks like a double descent, but it is not the one described above. At the peak the network fits only 74% of its training images, far from interpolation, and the rise coincides with the warm-up of the OneCycle schedule, whose learning rate reaches its maximum at epochs 6 and 7 (panel c). The validation curve keeps oscillating while the learning rate stays high and settles as it anneals; the fusion model, trained with the same schedule, oscillates over the same epochs. We therefore read it as optimisation instability at a large learning rate, not as a capacity effect. A likely contributor, which we did not isolate, is BatchNorm [15]: validation uses running statistics that lag behind weights that are changing quickly, so the validation pass sees a slightly different network from the one the training loss measures.

ConvNeXt-Tiny shows a much smaller bump (panel d): the mean validation loss rises from 0.112 at epoch 4 to 0.133 at epoch 5 and returns to 0.113 at epoch 6, and validation accuracy dips in four of the five folds (from 96.91% to 96.58% on average). Here training accuracy is already 98.8%, close to the regime where

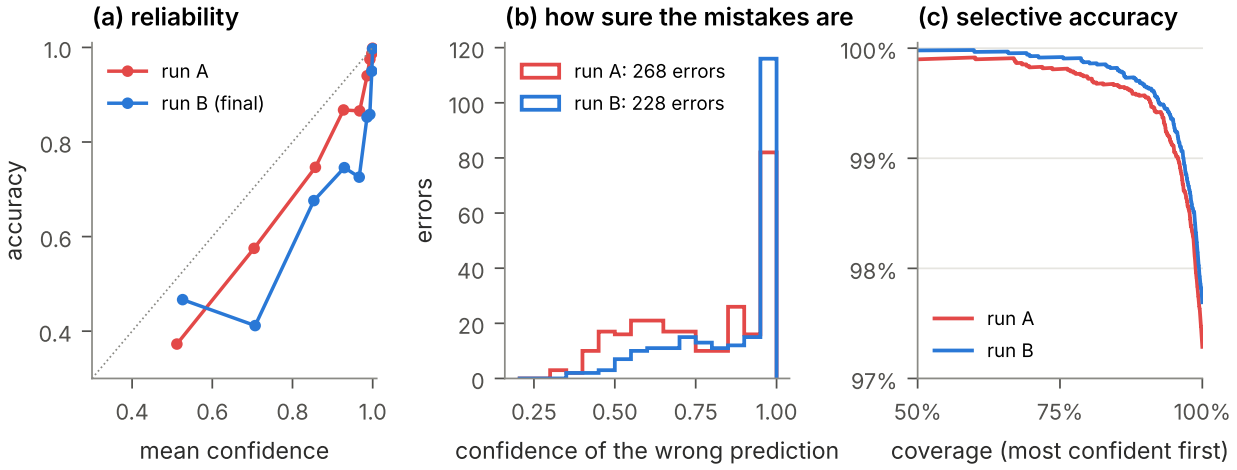


Figure 14. More accurate and more over-confident. Out-of-fold predictions of run A (lowest-validation-loss epoch) and run B (last epoch, final model). (a) Reliability diagram: both runs are over-confident below 99% confidence. (b) Confidence of the wrong predictions: run B makes fewer mistakes, and more of them with near-certain probability, which is what raises its log loss. (c) Accuracy of the most confident fraction of images: run B is above run A at every coverage; at confidence ≥ 0.99 it still covers 95.3% of the images at 99.25% accuracy.

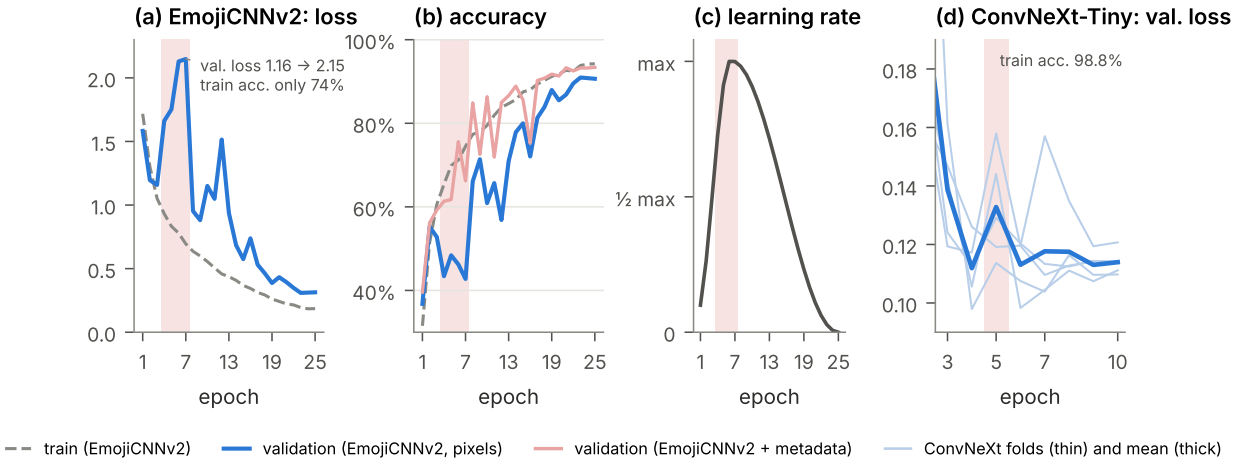


Figure 15. A second descent that is not (clearly) double descent. (a, b) EmojiCNNv2 on the stored split: validation loss rises and accuracy falls between epochs 3 and 7 (shaded) while training loss keeps decreasing and training accuracy is only 74%. (c) The rise coincides with the peak of the OneCycle learning rate. (d) ConvNeXt-Tiny, run A: a small bump at epoch 5, when training accuracy is already 98.8%, in the regime where epoch-wise double descent is reported [38], but too small to separate from noise.

epoch-wise double descent appears. The shape is consistent with it, but a 0.3-point dip from a single seed is within the epoch-to-epoch noise of a 1,976-image fold (standard error about 0.4 points), so we do not claim it. Model-wise double descent would need a sweep of width or depth under a fixed recipe; our ladder changes several factors at once and cannot be read that way. The practical lesson holds for both networks: stopping at the first minimum of the validation loss would have kept EmojiCNNv2 at 53% accuracy (epoch 3) instead of 91%, which is why the codebase never stops early under OneCycle, and it echoes the conclusion of Section 6.4 reached from the calibration side.

6.6 Did we overfit or underfit?

A model underfits when it cannot even fit its training data (high bias: training and validation accuracy are both low and close), and overfits when it fits the training data much better than new data (high variance: a large and growing gap). Table 9 puts the training and validation numbers of every model whose curves we

Table 9. Training versus validation at the checkpoint each run kept[V]. ConvNeXt values are the mean of the five folds at epoch 10 for a single model without test-time augmentation; the ensemble's out-of-fold accuracy is 97.69%.

Model (epoch kept)	Train acc.	Val. acc.	Gap	Train loss	Val. loss	Diagnosis
Baseline CNN, 30 Sept. (20 of 20)	74.96%	76.71%	−1.8	0.722	0.714	underfits
EmojiCNNv2 (23 of 25)	93.82%	90.96%	2.9	0.194	0.309	slight underfit
EmojiCNNv2 + metadata (25 of 25)	96.24%	93.38%	2.9	0.133	0.240	as above
ConvNeXt-Tiny, 5 folds (10 of 10)	99.92%	97.62%	2.3	0.004	0.114	benign fit

saved side by side. Training accuracy is measured on the fly during each epoch, with augmentation, dropout and stochastic depth active, so it slightly understates how well the network fits the clean training images.

The first CNN underfitted. Its training accuracy was 75%, no higher than its validation accuracy, and both were still rising when training stopped after 20 epochs. That is the signature of high bias: the network, its optimiser and its training budget were too weak to fit the data at all, and more regularisation or more data would not have helped. We fixed it with capacity and optimisation (BatchNorm, a fourth stage, AdamW with OneCycle), not with more data, and gained 14 points.

EmojiCNNv2 is still slightly bias-limited. It fits 94% of its training images and generalises to 91%, a gap of about three points. The gap is small and stable over the last epochs (Fig. 15b), so variance is not the main problem; the remaining errors come from what a 0.39 M-parameter network sees at 96 px. A larger pretrained model at higher resolution removed most of them (+6.1 points on the same 1,979 validation images), which is what one expects when the limit is bias rather than variance.

The final ConvNeXt fits the training set, and that is benign. By the last epoch it classifies 99.9% of its training images correctly with a training loss of 0.004, 27 times lower than its validation loss: in the classical sense it has memorised the training set. Yet its validation accuracy kept rising until the very last epoch, which is exactly why the last-epoch checkpoint beats the lowest-loss one (Section 6.4). The overfitting shows in the probabilities, not in the decisions: the network becomes over-confident on the few images it gets wrong, which raises the validation loss while accuracy still improves. Four further checks point the same way. The private leaderboard score (0.98218) is above the out-of-fold estimate (97.69%). The five folds agree within 0.4 points, the spread expected from sampling alone. Stronger regularisation (rotation and scaling augmentation with label smoothing, E7) lowered accuracy instead of raising it. And retraining on all the data, which would help a variance-limited model, changed the private score by only $+0.14$ and -0.08 points.

We did not overfit the validation set or the leaderboard. Every split groups exact duplicates, so no image is validated against a copy of itself; the final checkpoint rule (last epoch) involves no selection on validation data; the test set was never used to choose anything; and every model scores higher on the private half of the test set than on our validation estimate (Fig. 20). If anything, our validation protocol is conservative.

6.7 Error analysis and explainability

The 228 out-of-fold errors are not spread evenly. Only 28 of them cross an export family: accuracy is 96.6% inside the 72 px family (5,906 images), 99.7% in the 160 px family (1,486) and 99.2% in the 240 px family (2,487) (Fig. 16). The network never sees the native size, since every image is resampled to 224 px, yet it separates the three families from the rendering alone and spends its errors on vendors that draw alike within a family. The smaller models show the trend: the September CNN made 176 cross-family errors, EmojiCNNv2 57, and the fusion model, which is told the family by the metadata, 6.

The confusion matrix (Fig. 17) names the pairs: Apple predicted as Google 37 times, WhatsApp as Apple 36, Facebook as Apple 28, Google as Apple 17, Apple as Samsung 13 and Samsung as Apple 12. Apple sits

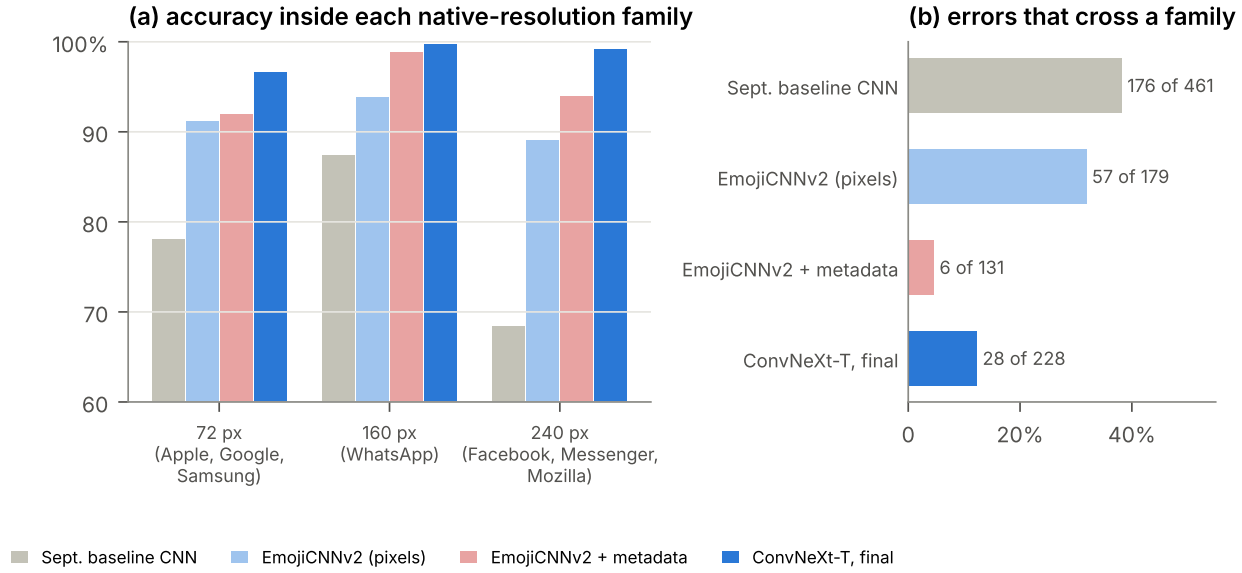


Figure 16. Errors stay inside an export family. (a) Accuracy inside each native-resolution family; the three CPU models are scored on the stored split, the final model out-of-fold on all images. (b) Share of errors that assign an image to a vendor of another family. Only the fusion model, which receives the file format explicitly, crosses families less often than the final pixel-only model.

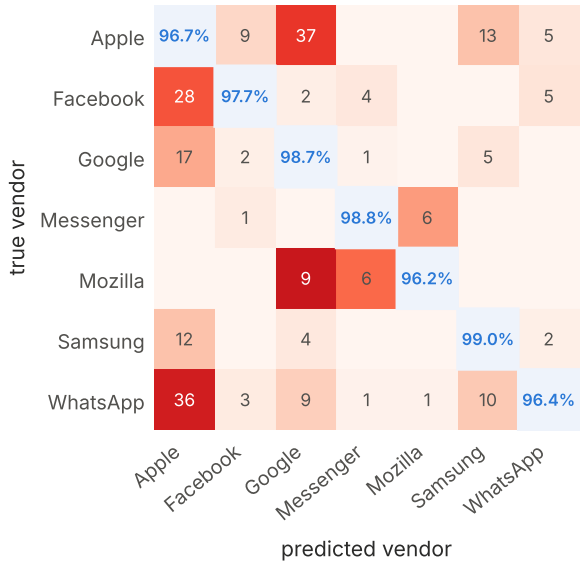


Figure 17. Out-of-fold confusion matrix of the final model. The diagonal gives per-vendor recall; off-diagonal cells give error counts, shaded by their share of the row. Apple is the hub of the confusions: WhatsApp's set was modelled on Apple's [39], and Apple, Google and Samsung share the 72 px export. Facebook, Messenger and Mozilla form the second cluster (240 px), Messenger having used Facebook's set since October 2017 [27].

at the centre for a documented reason: when WhatsApp released its own set in October 2017, Emojipedia described it as "a unique set (albeit heavily inspired by the Apple Color Emoji font)" [39], and Apple, Google and Facebook share conventions for faces. Figure 18 shows the most confident mistakes. Several are near-identical designs across related vendors; others are flags, symbols or simple shapes with almost no vendor-specific drawing; a few look to us like drawings of the vendor the model predicted, which may point to label noise that we cannot verify.

To check what the networks attend to we applied Grad-CAM [40] to the last convolutional stage of EmojiCNNv2 (Fig. 19). Its attention falls mostly on the drawing itself (outlines, eyes, shaded regions); on flat designs such as flags or a lozenge it spreads along the edge of the shape, where anti-aliasing differs between vendors. We see no case where it rests on the empty canvas, which is what the preprocessing was meant to ensure. The small model's confusions follow the same groups as the final model's: Facebook with Messenger, Mozilla with those two, Apple with Samsung (Appendix C).

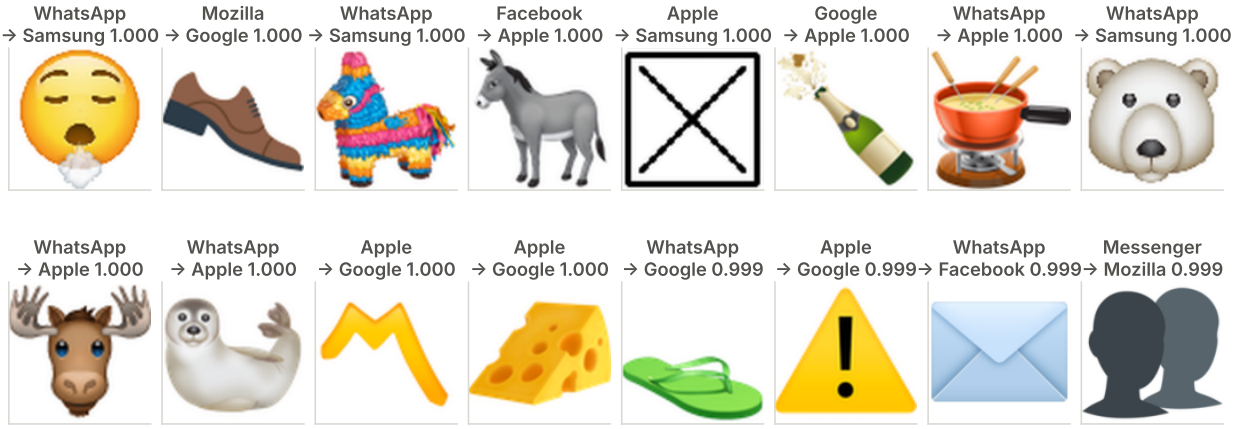


Figure 18. The sixteen most confident out-of-fold mistakes of the final model (true vendor → predicted vendor, confidence). Most are look-alike designs or images with little vendor-specific detail.

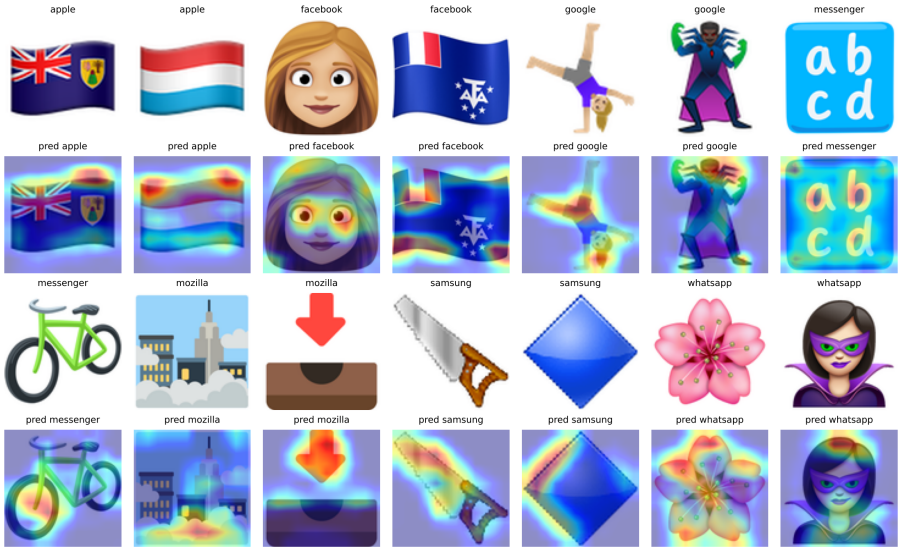


Figure 19. Grad-CAM on EmojiCNNv2 for correctly classified validation images, two per vendor. Top rows: input; bottom rows: Grad-CAM heat map of the predicted class. Attention concentrates on contours, eyes and shading, and on shape edges for flat designs.

6.8 Kaggle leaderboard

The leaderboard splits the test set into a public half, shown during the competition, and a private half (about 50% of the test images) that decides the final ranking. Table 10 lists the team's entries with both scores, read from the competition page after it closed. Every file produced by this codebase scores *above* its validation estimate on the private half, by 0.5 to 2.2 points (Fig. 20). For the ConvNeXt files part of this is expected, because the out-of-fold estimate scores single fold-models while test predictions average five; for every model it means the protocol did not flatter us.

The team's two selected entries at the close were our final model and Erdem's E15, a soft vote of three ConvNeXt-Tiny models trained on all training data, one at 224 px with bicubic resizing and two at 144 and 216 px with nearest-neighbour resizing, which are exact integer upscalings of the 72 px files and follow the same exact-pixel logic as our augmentation. E15 scored 0.98238 on the private half, our final model 0.98218, a difference of about one image out of roughly 4,940. The team finished fifth of 18 teams on the private leaderboard, with the same score as the fourth-placed team. The two leading teams scored 0.99979 and 0.99919; we do not know their methods.

Table 10. Kaggle scores (accuracy) of the team's entries, read on 27 October 2026 after the competition closed. Kaggle ranks a team by the better private score of its two selected entries (✓). Two early test entries (private 0.83259 and 0.77874) and a duplicate upload are omitted.

Entry	Model	Val.	Public	Private	
✓ `convnext224_last`	final model (run B), this paper	97.69%	0.98319	0.98218	V
`convnext224_AB_10models`	10-checkpoint average (ablation)	97.70%	0.98279	0.98198	V
`emojicnnv2_fusion`	EmojiCNNv2 + file metadata	93.38%	0.93824	0.94615	V
`emojicnnv2`	EmojiCNNv2, pixels only	90.96%	0.90747	0.91700	V
`adam_baseline`	September baseline CNN	76.71%	0.77546	0.78906	V
✓ `E15_ens_full_3models`	3 ConvNeXt (224 bicubic, 144/216 nearest)		0.98117	**0.98238**	R
`E13v_ens_val_3models`	ensemble E8 + E12b + E13		0.97772	0.97955	R
`E9_ens_convnext128_224`	ConvNeXt 128 + 224 px	97.93%	0.97671	0.97874	R
`E10b_convnext_tiny_224_full`	E8 retrained on all data		0.97833	0.97773	R
`E11_ens_full_128_224`	E9 retrained on all data		0.97914	0.97793	R
`E8_convnext_tiny_224`	ConvNeXt-Tiny 224 px	97.57%	0.97408	0.97631	R
`submission2` (Lina)	(no description recorded)		0.96719	0.96983	R
`dinov2_convnext_ensemble`	DINOv2 [23] + ConvNeXt ensemble		0.96881	0.96477	R
`submission` (Charles)	ResNet18 fine-tuned, 160 px		0.95889	0.96396	R
`dinov2_finetuned` (Lina)	DINOv2 fine-tuned		0.95586	0.95607	R
`E3_simple_cnn_white`	simple CNN, white compositing	93.62%	0.93642	0.94331	R

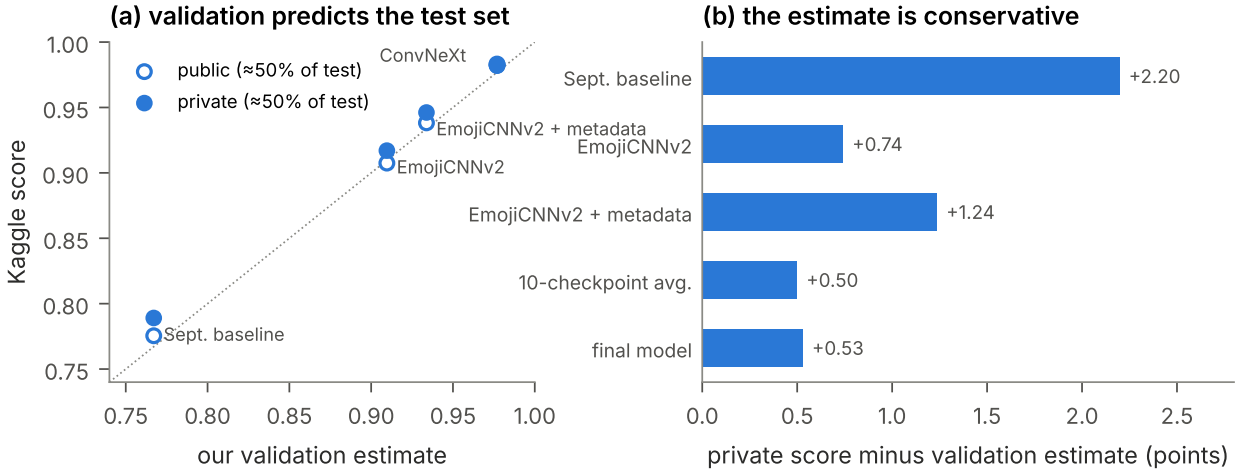


Figure 20. Validation estimates against the hidden test set, for the five files produced by the codebase. (a) Kaggle accuracy against our validation estimate; the dotted line is $y = x$. (b) Private score minus validation estimate: positive for every model, so the estimates are, if anything, conservative.

6.9 What did not work

- **Frozen ImageNet features**: 77.48%[R], below a 0.4 M-parameter CNN trained from scratch.
- **Resampling augmentation and label smoothing**: minus 1.5 points on ConvNeXt-Tiny at 128 px[R]. Three things changed at once (rotation and scaling, label smoothing, 15 instead of 10 epochs), so the run does not say which one hurt, but it agrees with the exact-pixel argument of Section 3.3.
- **Voting with weaker members**: adding ResNet18 or the small CNN to the ConvNeXt ensemble lowered accuracy to 97.3 to 97.4%[R]; a DINOv2 + ConvNeXt ensemble scored 0.96477 private against 0.98218 for our ConvNeXt alone (Section 4.4); and averaging in our own earlier snapshots did not help (Section 6.4). Averaging helps between comparably strong models.
- **The September training budget**: constant Adam 10^{-3} for 20 epochs left the first CNN underfitting at 75% training accuracy[V]; BatchNorm and OneCycle, not more parameters, closed most of the gap.

- **Keeping the lowest-validation-loss checkpoint**: 0.40 points below keeping the last epoch[V].

7 AI and code

7.1 Codebase and reproducibility

The submitted codebase is a small Python library (`src/emoji_provider`: data, EDA, models, training, evaluation, metadata model, submission writer) with scripts and one notebook as entry point, built on PyTorch [41], `timm` [3] and scikit-learn [29]. `README.md` gives the exact commands of both GPU runs and of the Colab, Databricks and laptop variants; running it on a new machine needs only the Kaggle archive and the codebase zip. Every script checks its inputs with assertions (label columns, ID sets, train/test disjointness, duplicate isolation across splits, submission format and a round trip through CSV). Each run writes its configuration, device, kept epoch and package versions, its training history, its validation and test probabilities and its figures, so every number and figure in this paper is regenerated from saved outputs by `scripts/build_report_tables.py` without retraining; the full console log of both GPU runs is kept. Seeds are fixed, the validation split and the fold assignment are files, six unit tests pass, and a smoke mode exercises the whole GPU path on a laptop in minutes. The strongest reproducibility check was not planned: run B reproduced run A epoch by epoch to the last decimal.

```
notebooks/Emoji_Provider_Classification.ipynb                entry point (RunPod / Colab / Databricks / laptop)
src/emoji_provider/  data.py  eda.py  models.py  train.py  evaluate.py  metadata_model.py
submit.py
scripts/  run_cpu_experiments.py  train_backbone.py  make_final_submission.py
build_report_tables.py
data/  split_seed42.csv  baseline_sept30/best_model.pt
outputs/  eda/  cpu/  convnext224/ (run A)  convnext224_last/ (run B)  submissions/
runpod_training_logs.txt
README.md  RUNBOOK.md  AI_USAGE.md  requirements.txt  requirements-gpu-runtime.txt  tests/
```

7.2 Compute

EmojiCNNv2 trains at 143 s per epoch on two CPU cores (25 epochs). ConvNeXt-Tiny at 224 px takes 2.4 to 2.9 minutes per fold, 13.0 minutes for the five folds, on an NVIDIA RTX 4090 rented on demand from RunPod at $0.89 per hour; each run used about 16 minutes of pod time including PNG decoding, inference with test-time augmentation and figures, and both runs together cost about $1.10. A single 224 px model took about 2 h 10 on an Apple M4 laptop[R]. Package versions are listed in Appendix D.

7.3 Use of AI tools

As the course allows, AI assistants were used for code and experiments, and, as the teacher allowed in class, for drafting this paper; every use and its prompts are listed in `AI_USAGE.md`. The September baseline notebook was generated and debugged with OpenAI Codex. Claude (Anthropic) wrote the submitted codebase, ran the CPU experiments, drove the two GPU runs on a pod created by the team, generated the tables and figures from the saved outputs and drafted the text; the team chose the experiments, reviewed every claim against the outputs and is responsible for the content.

8 Conclusion

Recognising who drew an emoji is a style problem, and the project taught us as much about the data as about models. Looking at the files before the pixels exposed a shortcut worth 87% accuracy without any drawn pixel; removing it made every later comparison about what the network sees. On the modelling side the lessons were plain. A small CNN trained properly beats hand-crafted features by ten points. Pretrained features help only once the whole backbone is fine-tuned. Input resolution mattered more than the architecture. Augmentations that resample the image erase the detail that identifies the vendor. And at 97% accuracy the rule for choosing a checkpoint is worth 0.4 points: the last epoch of a OneCycle schedule is both the simplest and the best choice we found, because validation loss rewards hedging while

accuracy rewards decisions. The final ConvNeXt-Tiny ensemble reaches 97.69% out-of-fold and 0.98218 on the private leaderboard, separates the three export families from pixels alone, and fails on documented look-alikes, the behaviour of a model that learned style rather than file format.

Limitations. The reported rows of Table 6 come from single runs on a different split and carry no error bars. Our architecture comparison rests on those runs. We did not train at the native 240 px, did not retrain the final recipe on all training data, and could not verify the labels of the most confident errors.

Future work. Train at native resolution with integer (nearest-neighbour) upscaling of the 72 px files, which Erdem's best entry suggests is promising; try a two-stage classifier that predicts the export family and then the vendor within it, an explicit version of what the network does implicitly; run a seed study to put error bars on the architecture comparison; and replace the equal-weight vote by a learned combiner over several backbones, trained only on out-of-fold predictions. Our evidence suggests the last gain would be small: an ensemble with a weaker DINOv2 model lost 1.7 points on the private leaderboard.

References

[1] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11986, 2022.

[2] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[3] Ross Wightman. PyTorch image models. `https://github.com/huggingface/pytorch-image-models`, 2019. Model `convnext_tiny.fb_in1k`.

[4] Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. Image style transfer using convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2414–2423, 2016.

[5] Sergey Karayev, Matthew Trentacoste, Helen Han, Aseem Agarwala, Trevor Darrell, Aaron Hertzmann, and Holger Winnemoeller. Recognizing image style. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2014.

[6] Zhangyang Wang, Jianchao Yang, Hailin Jin, Eli Shechtman, Aseem Agarwala, Jonathan Brandt, and Thomas S. Huang. DeepFont: Identify your font from an image. In *Proceedings of the 23rd ACM International Conference on Multimedia*, pages 451–459, 2015.

[7] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.

[8] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012.

[9] Björn Barz and Joachim Denzler. Do we train on test data? Purging CIFAR of near-duplicates. *Journal of Imaging*, 6(6):41, 2020.

[10] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 886–893, 2005.

[11] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.

[12] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[13] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[14] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[15] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *PMLR*, pages 448–456, 2015.

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[17] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, pages 6105–6114, 2019.

[18] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[19] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[21] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[22] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[23] Maxime Oquab, Timothée Darcet, Théo Moutakanni, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.

[24] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[25] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[26] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[27] Jeremy Burge. Facebook discontinues Messenger emojis. Emojipedia blog, 2 October 2017, https://blog.emojipedia.org/facebook-discontinues-messenger-emojis/, October 2017.

[28] Rafael Müller, Simon Kornblith, and Geoffrey E. Hinton. When does label smoothing help? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[29] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[30] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[31] Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, volume 11006 of *Proceedings of SPIE*, 2019.

[32] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, pages 646–661, 2016.

[33] Paulius Micikevicius, Sharan Narang, Jonah Alben, et al. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

[34] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330, 2017.

[35] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[36] Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2018.

[37] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116, 2019. doi: 10.1073/pnas.1903070116.

[38] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. In *International Conference on Learning Representations (ICLR)*, 2020.

[39] Jeremy Burge. WhatsApp releases its own emoji set. Emojipedia blog, 23 October 2017, https://blog.emojipedia.org/whatsapp-releases-its-own-emoji-set/, October 2017.

[40] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[41] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

A The team's 13 October experiment log

Table 11. Erdem's experiment series on an 80/20 split[R]. Public scores where the experiment was submitted.

Id	Model and method	Val. acc.	Public
E1a	random forest on file metadata (size, mode, hidden colour)	65.84%	
E1b	RBF SVM on 512-bin colour histogram + HOG, 64 px	81.28%	
E2	simple CNN (32-64-128), naive RGB load, 96 px	92.11%	
E3	same CNN, alpha-composited on white	93.62%	0.93642
E4	ResNet18, frozen ImageNet features, 128 px	77.48%	
E5	ResNet18, fine-tuned, 128 px	93.72%	
E6	EfficientNet-B0, fine-tuned, 128 px	93.12%	
E6b	ConvNeXt-Tiny, fine-tuned, 128 px	95.90%	
E7	as E6b + rotation/scale augmentation + label smoothing	94.38%	
E8	ConvNeXt-Tiny, fine-tuned, 224 px	97.57%	0.97408
E9	soft vote E6b + E8	97.93%	0.97671

B Who tried what

Table 12. Methods tried by each team member, with their best recorded score. The repository has one folder per member (`members/`) that holds these notes and, where available, the code.

Member	Methods	Best recorded score
Adam Zeraiki	First CNN (30 September, with OpenAI Codex); EmojiCNNv2 and the metadata diagnostic; ConvNeXt-Tiny 5-fold runs A and B and the 10-checkpoint average; DINOv2 + ConvNeXt combination; this paper	97.69% OOF, 0.98218 private (final model)
Erdem Sanver	Experiments E1a to E15: metadata random forest, HOG + colour SVM, simple CNN with and without compositing, ResNet18 frozen and fine-tuned, EfficientNet-B0, ConvNeXt-Tiny at 128 and 224 px, augmentation study, ensembles, retraining on all data, nearest-neighbour upscaling	97.93% validation (E9); 0.98238 private (E15, team best)
Lina Oumhand	DINOv2 fine-tuned; two earlier entries (no description recorded)	0.96983 private (`submission2`)
Charles Montluc	ResNet18 pretrained on ImageNet, fine-tuned on 160 px RGB images	0.96396 private

C Per-vendor results of the CPU models

Table 13. Recall per vendor on the stored 1,979-image split[V].

Model	Apple	Facebook	Google	Messenger	Mozilla	Samsung	WhatsApp	Macro F1
September baseline	0.765	0.693	0.940	0.421	0.744	0.751	0.817	0.738
EmojiCNNv2	0.897	0.834	0.964	0.917	0.878	0.972	0.881	0.907
EmojiCNNv2 + metadata	0.920	0.910	0.953	0.926	0.841	0.986	0.922	0.926

D Environment

CPU experiments: Python 3.13, torch 2.14.1, torchvision 0.29.1, timm 1.0.30, scikit-learn 1.9.1, numpy 2.5.3, pandas 3.0.5, Pillow 12.3.0 (pinned in `requirements.txt`). GPU runs: RunPod PyTorch 2.8.0 image, torch 2.8.0+cu128, NVIDIA driver 570.169, RTX 4090 24 GB, with scikit-learn and timm installed from PyPI on 17 October 2026. The September baseline was trained with torch 2.11.0+cu128; re-scored with the current versions it reproduced 1,518 / 1,979 correct exactly.